

Afillar Insurance APIs

UAT Deployment Plan

MC to UAT: inventory, sequence, verification and rollback

Version 1.1

02-Oct-2026

Prepared for the Afillar release and platform teams

Table of Contents

Table of Contents	2
1. Purpose and Scope	3
1.1 Assumptions and Open Items	3
2. What Has to Be Deployed	4
3. Group A — Platform Prerequisites in UAT	5
4. Group B — Product Configuration	6
4.1 Contents Per Product	6
4.2 How	6
5. Group C — iTables	7
5.1 Row Sources	7
5.2 Change Before Loading	7
5.3 If PA001 Is Not in UAT	8
6. Group D — Output Templates	9
7. Group F — Environment Settings and Access	10
8. Group E — iComposer Code	11
8.1 How	11
9. Deployment Sequence	12
10. Verification in UAT	13
11. Rollback	14
12. Risks, Dependencies and Go / No-Go	15
12.1 Go / No-Go Checklist	15

1. Purpose and Scope

This document answers two questions for the Afillar Insurance Company insurance APIs: **what has to be deployed**, and **how the deployment from the MC tenant to UAT is done and verified**. Everything described here currently exists only in the MC (development) tenant. Nothing has been deployed to UAT: this is the plan.

Item	Value
Source tenant (MC)	tenant code <code>uponly</code> , profile <code>portal:uponly</code> , gateway <code>https://portal-gw.insuremo.com</code>
Target tenant (UAT)	to be confirmed: tenant code, <code>imo</code> profile and gateway of UAT (placeholders <code><UAT></code> below)
Scope	10 common functions, 12 active APIs, 12 iTables with their rows, product configuration of 7 products, 14 output templates, environment settings, access
Not in scope	PA001 (must already exist in UAT or is not needed), the retired API <code>UPCancelPolicy</code> , test data, scripts and reports
Release	Insurance API kit v1.1 (insurer <code>CarrierCode</code> / <code>CarrierName</code> added), specification v1.1, policy model v0.4

1.1 Assumptions and Open Items

#	Assumption / open item	Owner
1	UAT is a separate InsureMO tenant reachable with <code>imo</code> (profile name, tenant code and gateway to be provided).	Project / platform team
2	The deployer has admin rights in UAT for iComposer, iTables, product configuration, Output Service and code tables.	Platform team
3	PA001 exists in UAT (the new products were created as copies of its structure). If PA001 does not exist in UAT, section 5.3 lists what must be created first.	Platform team
4	The partner (Sohar Bank) API users for UAT, with the Checker and Approver roles, are created by the platform team; credentials are not part of this package.	Platform team
5	Ids of products, data tables, templates and data-dictionary objects differ per tenant and are re-resolved during the deployment, never copied.	Deployer
6	Write access to configuration is allowed in UAT for the deployer account (it is required to load the tables).	Platform team

2. What Has to Be Deployed

Six groups, A to F. Each group is described in sections 3 to 8; the order in which they are deployed is in section 9 (A, C, E, B, D, F).

#	Group	Contents	Count	Method
A	Platform prerequisites	Code tables, field library, PA001 base objects, PubBranch value 1, API users	see 3	Platform admin
B	Product configuration	7 tech + 7 market products, data dictionary bindings, 7 rating definitions, cancellation delta fields	14 products	<code>build_product.py</code> , <code>rating_build.py</code> , <code>fix_endo_fields.py</code>
C	iTables	12 UP_ tables (definition + rows)	12 tables, about 550 rows	<code>imo config</code> <code>datatable create +</code> <code>record-batch-save</code>
D	Output templates	Application form and certificate templates per product	14 templates	Output Service upload (<code>os_gen.py</code>)
E	iComposer code	10 functions, 12 active APIs (13th stays inactive)	22 files	<code>imo icomposer push</code> <code>current</code>
F	Environment settings and access	3 environment specific config values, channel row, gateway route, API users and roles	-	Row changes + platform admin

3. Group A — Platform Prerequisites in UAT

These must exist before anything else is loaded. Check each one in UAT; create it only if it is missing.

Prerequisite	Why	How to check
PA001 and PA001_TECH products	The new products were built from its structure; the rating definitions were copied from PA001_PREM_CALC	<code>imo config product list --code PA001 --profile <UAT></code>
Tenant tables used by the code and as code tables: BranchCode (001), CustomerType, GenderCode, IdType, InsMaritalStatus, Plan (ids 1-5), PremiumMode (1, 2, 3), Relationship, ResidentStatus, Salutation	Code tables behind the validation rules and the data dictionary bindings; also listed by Master Data	<code>imo config datatable by-name --name <T></code> and <code>imo config codetable data --name <T></code> ; values must equal MC (see the specification section 5.1)
Code table PubBranch with value 1	Cancellation endorsement validation checks OrgCode against it	<code>imo config codetable data --name PubBranch</code>
Field library entries of the data dictionary (ApplicationNo, ApplyDate, BranchCode, CIFNumber, DiscountAmount, StaffId, UserName, PlanId, PolicyTerm, PremiumModeCode, ProductSubcode, ChannelCode, Payment* ...)	Fields bound to the new products; ids in <code>fields_created.json</code> are MC ids	<code>policy_field.py search-fields --name <Field></code> ; create missing ones with <code>fields.py</code>
Cancellation endorsement rating definition ("Cancellation Endorsement", tenant level)	Used by <code>calculateEx</code> ; identical in the other tenants checked	<code>rating_api.py definition-load</code>
API users, roles (Checker, Approver), gateway route to the iComposer APIs, token (CAS) endpoint for the partner	Access to the APIs	Platform admin

4. Group B — Product Configuration

For each of the seven product codes the following objects are created in UAT with the same names; the ids will differ.

Product code	Tech product	Market product	MC tech id	MC market id	Rating definition
UPPA001	UPPA001_TECH	UPPA001	1188461992	1188462026	UPPA001_PREM_CALC
TL001	TL001_TECH	TL001	1188461622	1188499145	TL001_PREM_CALC
FP001	FP001_TECH	FP001	1188461381	1188461427	FP001_PREM_CALC
HC001	HC001_TECH	HC001	1188461353	1188461371	HC001_PREM_CALC
CI001	CI001_TECH	CI001	1188499250	1188461745	CI001_PREM_CALC
LP001	LP001_TECH	LP001	1188461838	1188461893	LP001_PREM_CALC
DH001	DH001_TECH	DH001	1188461433	1188461520	DH001_PREM_CALC

4.1 Contents Per Product

- Tech product with the policy tree: Policy, Product (PolicyLob), Insured R00001 (PolicyRisk), Plan, PolicyCoverage, Beneficiary; the product-level field bindings (24 policy, 17 tech, 8 risk, 4 plan, 6 coverage, 4 beneficiary fields) and the code-table links (`step_codetables`).
- Market product bound to the tech product and the premium rating definition `<CODE>_PREM_CALC` (copy of `PA001_PREM_CALC` bound to the market product, patched to read the `UP_` tables).
- Cancellation delta fields on the tech product (`StandardGrossPremiumDelta`, `AnnualPremiumDelta`, `ShortRate` on the objects the platform reports as missing): `fix_endo_fields.py <TAG>` run against a test policy of each product (it binds only on the seven tech products).
- No VAT binding is created (there is no VAT in this solution).

4.2 How

1. `build_product.py spec_<CODE>.json all` with `GW`, `TOK` and `B` pointing to UAT (steps tech, tree, fields, codetables, market).
2. `rating_build.py <CODE> --confirm`, then `patch_rating.py` / `patch_rating2.py` (re-point the formulas to the `UP_` tables).
3. Create and issue one test policy per product (after group C and E), run `fix_endo_fields.py <TAG>` until the platform calculation passes ("CALCULATE OK").
4. Record the new product ids (`prod_info.py` writes `products.json`).

5. Group C — iTables

Create the tables in this order (the order does not matter technically, but the checks in section 8 assume it). Rows are written with `record-batch-save` in batches of 40; the primary key `Id` must be unique. The MC row files are in `tables/ (r_*.json)`; they must be re-read from MC just before the deployment so that the latest rows (for example the insurer rows) are included. The `DataTableId` inside the rows is a tenant id and must be replaced by the UAT table id (`uptab.py` does this).

Table	Rows (MC)	Columns	Environment note
UP_Channel	1	ChannelCode, Description, IsActive	Environment specific rows: channel(s) of the UAT partner
UP_Carrier	1	CarrierCode, CarrierName, IsActive	Environment specific rows: none
UP_ProductMaster	11	Description, LOB, IMOProductCode, IsRenewable, IsActive, IsFlatRate, Premium, Su	Environment specific rows: none
UP_ProductRule	11	ProductCode, ProductSubcode, PremiumModes, MinTerm, MaxTerm, MinAge, MaxAge, Max	Environment specific rows: none
UP_ProductPlanRelation	57	IMOProductCode, IMOPlanCode, IMOPlanName, CarrierCode, CarrierProductCode, Carri	Environment specific rows: none
UP_PlanBenefitRelation	148	IMOProductCode, IMOPlanCode, CarrierCode, CarrierProductCode, CarrierPlanCode, B	Environment specific rows: none
UP_DocTemplate	22	DocumentCode, TemplateName, FileName, ProductCode, ProductSubcode, DocType	Environment specific rows: none
UP_DocWording	27	ModeCode, Key, Value	Environment specific rows: none
UP_BenefitText	21	ProductCode, BenefitCode, NameEn, NameAr	Environment specific rows: none
UP_FeedColumn	29	Sequence, Heading, Source	Environment specific rows: none
UP_FieldRule	138	Api, Path, Label, Required, DataType, MinLength, MaxLength, Pattern, MinValue, M	Environment specific rows: none
UP_ApiConfig	81	Api, Scope, Key, Value	Environment specific rows: AllowedHosts, GatewayUrl, PrintUrl

5.1 Row Sources

Table	Source of the rows
UP_ApiConfig	<code>tables/r_cfg*.json</code> , <code>r_UP_ApiConfig_*.json</code> (about 81 keys) — or export from MC with <code>record-list</code>
UP_FieldRule	<code>tables/r_UP_FieldRule_*.json</code> , <code>r_carrierrule_0.json</code> (138 rows)
UP_ProductMaster / ProductPlanRelation / PlanBenefitRelation / ProductRule / DocTemplate	<code>tables/b_UP_*</code> and <code>rows_UP_*.json</code>
UP_DocWording, UP_BenefitText	<code>r_UP_DocWording_*</code> , <code>r_wordA_*</code> , <code>r_UP_BenefitText_*</code> , <code>r_benA_*</code>
UP_FeedColumn	<code>r_feedcol*.json</code> , <code>r_carrierfeed_0.json</code> (29 rows)
UP_Channel / UP_Carrier	<code>r_channel_0.json</code> , <code>r_carrier_0.json</code> (SOHAR = Sohar Bank; AFIC = Afillar Insurance Company)

5.2 Change Before Loading

- `UP_ApiConfig`: set `AllowedHosts`, `GatewayUrl`, `PrintUrl` to the UAT gateway (section 7).
- `UP_Channel`: the UAT channel code(s) of the partner.
- Everything else is loaded unchanged.

5.3 If PA001 Is Not in UAT

The solution does not read or change PA001 at run time, but the product structure and the rating were derived from it. In that case the platform team must provide the PA001 structure first, or the products must be built from the product specs without the copy (`build_product.py` creates the tree itself; only the rating definitions are copied from `PA001_PREM_CALC`).

6. Group D — Output Templates

The application form and the policy certificate PDFs are produced by Output Service templates named in `UP_DocTemplate`. They must exist in UAT under the same names. The templates live in the MC Output Service (the generated HTML working copies are not kept in the build folder; the images are in `assets/`): export them from MC, or rebuild them with `make_templates2.py` from the form designs.

#	Template name	Used for
1	UP_CI001_APPLICATION_EN	Application form
2	UP_CI001_POLICY_EN	Policy certificate
3	UP_DH001_APPLICATION_EN	Application form
4	UP_DH001_POLICY_EN	Policy certificate
5	UP_FP001_APPLICATION_EN	Application form
6	UP_FP001_POLICY_EN	Policy certificate
7	UP_HC001_APPLICATION_EN	Application form
8	UP_HC001_POLICY_EN	Policy certificate
9	UP_LP001_APPLICATION_EN	Application form
10	UP_LP001_POLICY_EN	Policy certificate
11	UP_TL001_APPLICATION_EN	Application form
12	UP_TL001_POLICY_EN	Policy certificate
13	UP_UPPA001_APPLICATION_EN	Application form
14	UP_UPPA001_POLICY_EN	Policy certificate

1. Export the 14 templates from the MC Output Service (or rebuild them with `make_templates2.py` from the form designs and the images in `assets/`).
2. Upload each template to the UAT Output Service with `os_gen.py <TEMPLATE_NAME> <html> <data.json> <out.pdf>` (uploads and generates a test PDF).
3. Compare the generated PDF with the MC PDF (layout, Arabic text, images).

7. Group F — Environment Settings and Access

UP_ApiConfig key	Meaning	UAT value
AllowedHosts	Host(s) the caller's token may be sent to (comma separated, https only)	UAT gateway host
GatewayUrl	Base URL of the platform gateway used for the endorsement REST calls	UAT gateway URL
PrintUrl	Print service URL used for the PDFs	UAT print service URL (same host as the gateway)

- `ChannelCode` of the partner: row in `UP_Channel`; the partner sends this code on every request.
- `OrgCode`: the branch code accepted by PubBranch in UAT (1).
- Test route versus runtime route: the APIs answer on `/platform/api-orchestration-test/v1/flow/up/...` after the push; the runtime route and its gateway policy (token, rate limiting, CORS) are set by the platform team.
- API users and roles: one Checker and one Approver user for the partner; the APIs do not check roles yet (deferred), so the separation is organisational until role access is built.
- Secrets: tokens and passwords never go into tables, collections or documents.

8. Group E — iComposer Code

Push in this order (callee before caller). `Compiled: yes` must be shown for every push; a failed compile stops the deployment.

Order	Asset	Kind / route	Lines	MC version	MC md5 (first 10)
1	UPDataUtil	Function	234	9	934ae12b56
2	UPSecurity	Function	115	4	a10494d853
3	UPErrorHandler	Function	99	9	3f42fef143
4	UPPlatformClient	Function	48	7	8e925c530f
5	UPValidator	Function	333	3	11a5b703d5
6	UPProductData	Function	201	4	7c8b8d30ca
7	UPShaper	Function	125	3	cae845cc83
8	UPDocBuilder	Function	194	3	7cf4a157c4
9	UPDocService	Function	104	4	8606b0a5d1
10	UPCancelService	Function	302	7	232c9e0e0b
11	UPGetPlans	API /up/get-plans	103	7	279231cf01
12	UPProposal	API /up/proposal	36	6	6a664918a9
13	UPUpdateProposal	API /up/update-proposal	57	9	12f3b6a05f
14	UPIssue	API /up/issue	50	9	467185f6ca
15	UPLoadPolicy	API /up/load-policy	35	7	2157203c2f
16	UPProposalReject	API /up/proposal-reject	42	9	01c301ef0c
17	UPDocGeneration	API /up/doc_generation	22	7	505d753dc5
18	UPApplicationDocGeneration	API /up/application_doc_generation	23	7	0818aeec73
19	UPCancelCheck	API /up/cancel-check	24	5	ab65f1fe0f
20	UPCancelApprove	API /up/cancel-approve	23	5	7ba502722a
21	UPListMasterTable	API /up/list-master-table	36	7	c7e9a27e99
22	UPCommissionQuery	API /up/commission-query	135	10	144617f061
23	UPFeedFile	API /up/generate-feedfile	177	9	191ab6f582

`UPCancelPolicy` (retired, inactive) is not deployed. `UPSecurity` and the `UP_Carrier` table are new in v1.1 and must be present before `UPProductData` and `UPShaper` (the shaper reads the insurer).

8.1 How

1. Export the current MC sources (`imo icomposer reload` into a clean workspace) so that the deployed code equals the code in the package.
2. Initialise a UAT workspace (`imo icomposer init`) and create the module group, then `imo icomposer create function` for each function (name, module id, group id, status 1, scope 1) and `create api` for each API (path and method as in the table above; POST except load-policy GET).
3. Copy the Groovy files into the UAT workspace and push each file in the order above: `imo icomposer push current <file> --profile <UAT>`.
4. Re-run `push` for a file that reports a conflict with `--prefer-local` only after comparing the server copy.

9. Deployment Sequence

Step	Action	Owner	Evidence
0	Freeze MC: no changes during the deployment; export the final MC rows and sources; run <code>snapshot.py</code> before in UAT (if PA001 exists)	Developer	snapshot files
1	Group A: confirm or create the prerequisites (section 3)	Platform team	checklist
2	Group C: create the 12 tables and load the rows with the environment values (section 5)	Developer	row counts equal MC (+ environment changes)
3	Group E: push 10 functions then 12 APIs in order (section 8); every compile OK	Developer	push log
4	Group B: create products, bindings, ratings (section 4)	Developer + platform	product list, rating loads
5	Group D: upload and test the 14 templates (section 6)	Developer	test PDFs
6	Group F: environment settings, channel row, users and roles, gateway route (section 7)	Platform team	config rows, user list
7	Smoke test: <code>carrier_check.py</code> (one flow per product) and <code>ref_flow.py</code> for one plan per product, Newman run of the Sohar Bank collection against UAT	Developer / QA	results files, Newman log
8	Security probe <code>security_probe.py</code> against UAT (after step 7 passes)	QA / security	probe results
9	Compare the UAT table row counts and the asset list with MC; <code>snapshot.py</code> after (PA001 unchanged)	Developer	comparison
10	Sign-off and go / no-go (section 12)	Project	signed checklist

10. Verification in UAT

Check	Expected	Tool
Every API answers 200 for a valid request, one product at a time	7 products x 8 calls pass, <code>CarrierCode</code> AFIC and the insurer name in every response	<code>carrier_check.py</code>
13-step flow for one plan per product incl. documents and cancellation	all steps 200, policy status 3 after APPROVE, refund returned	<code>ONLY_PLAN=1</code> <code>ref_flow.py <TAG></code>
Free look and pro-rata refund	day 15 = 100%, day 16 = 80% pro-rata (see the specification section 6.2)	<code>run_tests.py</code> all ST (cancellation by product block)
Error contract	400 / 404 / 409 bodies with <code>trace_id</code> ; no stack traces	<code>security_probe.py</code>
Security	no FAIL other than the known gateway findings	<code>security_probe.py</code>
PDFs	application form and certificate render for every product	collection folders / <code>os_gen.py</code>
Master data	code tables equal MC; <code>UP_Carrier</code> lists AFIC	<code>/up/list-master-table</code>
PA001 guard	PA001 and PA001_TECH hashes unchanged	<code>snapshot.py</code>

11. Rollback

- Code: keep the exported MC sources and the UAT sources of the previous push; roll back by pushing the previous files in the reverse order (APIs first, then functions).
- Tables: before loading, export the existing UAT rows (if any); rollback = delete the added rows (`record-batch-delete`) or `record-delete-all` for a table created by this deployment and restore the export.
- Products: new products are not used by anything else; if they must be removed, deactivate the market products (do not touch PA001).
- Templates: restore the previous template version in the Output Service.
- Because the APIs read tables at run time, a bad row can be corrected by a row change; no code deployment is needed for data fixes.

12. Risks, Dependencies and Go / No-Go

Risk / dependency	Impact	Mitigation
Ids differ per tenant (products, data tables, templates, DD objects)	Copying MC ids breaks the configuration	Re-resolve by name with the build scripts; never copy ids
PA001 or the base code tables missing in UAT	Products and rating cannot be built	Check group A first
Cancellation delta fields not bound	APPROVE returns 409 at calculation	Run <code>fix_endo_fields.py</code> per product (section 4.2)
PubBranch has no value 1 in UAT	APPROVE returns 409 at validation	Add the value or change the <code>OrgCode</code> override
Wrong host in <code>AllowedHosts</code> / <code>GatewayUrl</code> / <code>PrintUrl</code>	Cancellation and documents fail with UP-CONFIG / host error	Set the three keys to the UAT gateway before smoke tests
Role access not built	Checker / Approver separation is organisational only	Agree the interim control with the partner; build the access table later
Gateway findings (CORS, rate limiting)	Known platform level observations	Platform team before production
Per-plan collections not refreshed	Examples lack the insurer fields	Optional: regenerate after UAT

12.1 Go / No-Go Checklist

#	Criterion	Status	Signed by
1	Group A prerequisites confirmed in UAT		
2	12 tables loaded, row counts verified		
3	10 functions and 12 APIs pushed, all compiled		
4	7 products, bindings and rating definitions created		
5	14 templates uploaded, PDFs checked		
6	Environment keys, channel row, users, roles and route set		
7	<code>carrier_check.py</code> : all products pass		
8	Plan flow with cancellation passes for one plan per product		
9	Security probe: no unexplained FAIL		
10	PA001 snapshot identical		
11	Rollback tested on one table and one function		
12	Partner informed of the UAT URL and credentials process		